

INSPECTAIR

Design Description

System- und Software-Architektur

Projekt:

Version:

Datum:

Autoren:

Kurs:

Betreuer:

InspectAir – Luftqualitätsmessgerät

2.0

Februar 2026

Maximilian Liebl, Cedric Stroh, Jannis Messer

Mikrocomputertechnik 1, DHBW Ravensburg

Thomas Stingl (Airbus)

Dieses Dokument wurde unter Verwendung von KI-Tools (Claude, Anthropic) zur Überprüfung erstellt.

Inhaltsverzeichnis

1	Projektbeschreibung	2
1.1	Überblick	2
1.2	Kernfunktionen	2
1.3	Zielgruppe	2
2	System-Architektur	2
2.1	Hardware-Blockdiagramm	2
2.2	Schnittstellen-Übersicht	3
2.3	Kritische Hardware-Design-Entscheidungen	3
2.3.1	Stützkondensatoren (220 µF 25 V)	3
2.3.2	Level Shifter für LD2410C	3
2.3.3	BS170 MOSFET für Backlight	3
2.3.4	SoftwareSerial für MH-Z19C	4
3	Software-Architektur	4
3.1	Schichtenmodell	4
3.2	Verzeichnisstruktur	4
3.3	Datenstrukturen	5
3.4	Display-Architektur (LVGL 9)	5
3.5	State Machine	5
3.6	Verwendete Libraries	6

1 Projektbeschreibung

1.1 Überblick

InspectAir ist ein Luftqualitätsmonitor für Innenräume. Das System erfasst kontinuierlich relevante Luftqualitätsparameter und stellt diese auf einem 4-Zoll-TFT-Display dar. Durch Farbcodierung können kritische Werte auf einen Blick erkannt werden.

1.2 Kernfunktionen

- Messung von CO₂, Feinstaub (PM1.0/PM2.5/PM10), VOC, Temperatur und Luftfeuchtigkeit
- Echtzeit-Anzeige aller Messwerte auf 4-Zoll-Display (480×320 Pixel)
- Vierstufige Ampel-Farbcodierung (Grün/Gelb/Orange/Rot) nach WHO/UBA-Grenzwerten
- Fünf wählbare UI-Designs (Tree, Overview, Detail, Analog, Bubble)
- Präsenzerkennung via 24 GHz FMCW-Radar für automatisches Display-Management
- Energiesparmodi: Display-Dimming und Light Sleep

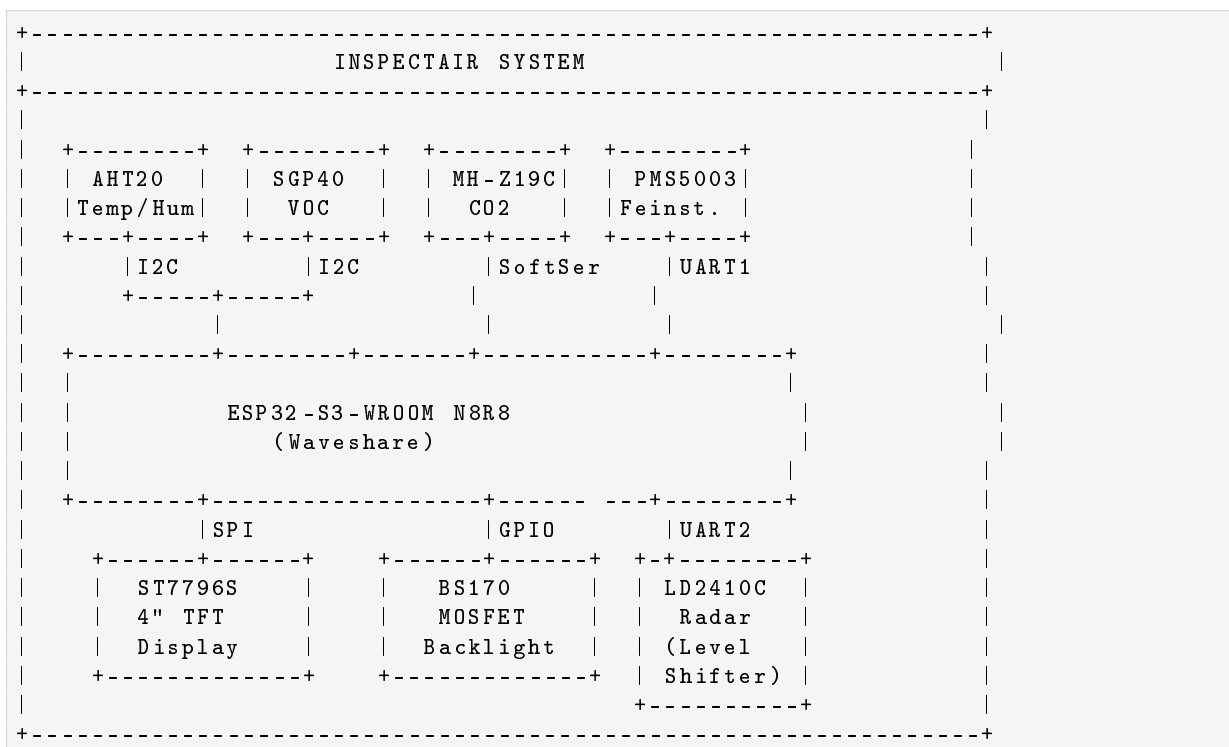
1.3 Zielgruppe

Das Gerät richtet sich an Privatpersonen und Büros, die ihre Raumluftqualität überwachen möchten. Typische Einsatzorte sind Wohnzimmer, Schlafzimmer, Büros und Klassenzimmer.

2 System-Architektur

2.1 Hardware-Blockdiagramm

Das System besteht aus folgenden Hardware-Blöcken:



2.2 Schnittstellen-Übersicht

Komponente	Interface	Spannung	ESP32 Pins	Besonderheit
ST7796S Display	SPI	3,3 V	11(MOSI), 12(SCLK), 14(CS), 9(DC), 46(RST), 3(BL)	480×320 Pixel
AHT20	I ² C (0x38)	3,3 V	8 (SDA), 18 (SCL)	Pull-ups 4,7 kΩ
SGP40	I ² C (0x59)	3,3 V	8 (SDA), 18 (SCL)	Parallel zu AHT20
MH-Z19C	SoftSerial 9600	5 V	4 (RX), 5 (TX)	220 µF Stützkond.!
PMS5003	UART1 9600	5 V	16 (RX), 17 (TX)	Molex-Stecker
LD2410C	UART2 256000	5 V	7 (RX), 6 (TX), 15 (OUT)	Level Shifter + 220 µF!
BS170 MOS-FET	PWM	3,3 V Gate	3	1 kΩ Gate-Widerstand
Button	GPIO	3,3 V	1	Interner Pull-up

2.3 Kritische Hardware-Design-Entscheidungen

2.3.1 Stützkondensatoren (220 µF 25 V)

Problem: MH-Z19C und LD2410C verursachen Stromspitzen beim Messvorgang, die zu Spannungseinbrüchen und Systemabstürzen führen.

Lösung: 220 µF Elkos (25 V) direkt an VCC/GND beider Sensoren puffern die Stromspitzen ab.

2.3.2 Level Shifter für LD2410C

Problem: Der LD2410C sendet TX-Signale mit 5 V-Pegel, die ESP32-GPIOs sind nur 3,3 V-tolerant.

Lösung: Bidirektionaler Level Shifter (TXS0108E) zwischen Radar-TX und ESP32-RX. Die RX-Richtung (ESP→Radar) kann direkt erfolgen, da der Radar 3,3 V akzeptiert.

2.3.3 BS170 MOSFET für Backlight

Problem: Das Display-Backlight benötigt mehr Strom als ein GPIO liefern kann, und PWM-Dimming ist gewünscht.

Lösung: BS170 N-Kanal MOSFET als Low-Side-Switch. Logic-Level-MOSFET schaltet bei 3,3 V durch. Gate über 1 kΩ an GPIO3 für PWM-Steuerung.

2.3.4 SoftwareSerial für MH-Z19C

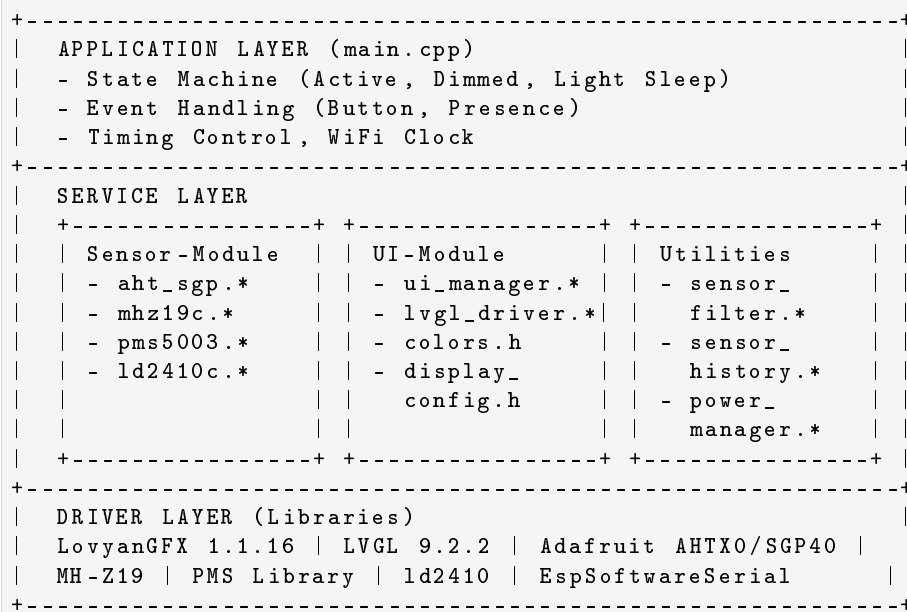
Problem: ESP32-S3 hat nur 2 Hardware-UARTs (UART0 für USB-Monitor, UART1 für PMS5003), aber 3 UART-Sensoren.

Lösung: EspSoftwareSerial (v8.2.0) für MH-Z19C bei 9600 Baud auf GPIO4/5. Die niedrige Baudrate ermöglicht zuverlässigen Betrieb per Software-UART.

3 Software-Architektur

3.1 Schichtenmodell

Die Software folgt einem dreischichtigen Architekturmodell:



3.2 Verzeichnisstruktur

```

inspectair/
  platformio.ini          # PlatformIO Konfiguration
  src/
    main.cpp              # Hauptprogramm, State Machine
    config/
      pins.h              # GPIO Pin-Definitionen
      sensor_types.h      # Sensor-Datenstrukturen
      colors.h            # Farben & Grenzwerte
      display_config.h    # Display-Konfiguration
      app_config.h        # Applikations-Konfiguration
    sensors/
      aht_sgp.cpp/.h      # AHT20 + SGP40 (I2C)
      mhz19c.cpp/.h      # MH-Z19C CO2 (SoftwareSerial)
      pms5003.cpp/.h      # PMS5003 Feinstaub (UART1)
      ld2410c.cpp/.h      # LD2410C Radar (UART2)
      sensor_filter.cpp/.h # Gleitender Mittelwert
      sensor_history.cpp/.h # 24h Datenspeicher
    display/
      ui_manager.cpp/.h   # LVGL 9 Multi-Screen UI
      lvgl_driver.cpp/.h  # LovyanGFX <-> LVGL Br cke

```

```

    fonts/                # Custom Fonts mit Umlauten
network/
  WifiClock.cpp/.h        # NTP-Zeitsynchronisation
  web_remote.cpp/.h       # Web-basierte Fernsteuerung
power/
  power_manager.cpp/.h    # Dimming & Light Sleep
lib/                      # Externe Libraries (PlatformIO)

```

3.3 Datenstrukturen

Die zentrale Datenstruktur **SensorReadings** (definiert in **sensor_types.h**) aggregiert alle Sensordaten:

```

typedef struct {
    AHT20_Data aht;        // Temperatur/Feuchte
    SGP40_Data sgp;        // VOC-Daten
    PMS5003_Data pms;      // Feinstaub (PM1.0/2.5/10)
    MHZ19C_Data mhz;       // CO2
    LD2410C_Data radar;    // Praesenz/Distanz
    uint32_t timestamp;    // Zeitstempel (millis)
} SensorReadings;

```

Jeder Sensor hat eine eigene Datenstruktur mit typisierten Feldern und Validierungsflags.

3.4 Display-Architektur (LVGL 9)

Die Display-Schicht besteht aus zwei Modulen:

lvgl_driver (Low-Level): Initialisiert LovyanGFX für den ST7796S-Controller und verbindet es als LVGL-Display-Treiber. Nutzt Dual-Buffer (je 480×32 Pixel) in PSRAM falls verfügbar. Stellt **lvgl_init()**, **lvgl_loop()** und **lvgl_set_brightness()** bereit.

ui_manager (High-Level): Verwaltet fünf verschiedene Screen-Designs:

Screen	Enum	Beschreibung
Tree	UI_SCREEN_TREE	Baum-Animation als Startbildschirm
Overview	UI_SCREEN_OVERVIEW	Große AQI-Anzeige + 2 Kacheln (minimalistisch)
Detail	UI_SCREEN_DETAIL	Kleine AQI + 4 Kacheln (alle Werte)
Analog	UI_SCREEN_ANALOG	Analoge Cockpit-Instrumente
Bubble	UI_SCREEN_BUBBLE	Dynamische Kreise (Modern Bubbles)

Custom Fonts (12/16/20/28/48 pt) mit deutschen Umlauten werden für die Darstellung verwendet.

3.5 State Machine

Das System verwendet eine State Machine für das Power Management:

```

+-----+
|  ACTIVE  | <-----+
| (100% BL) |         |
+-----+         | Praesenz

```

```

      | 60s keine Praesenz | erkannt
      v
+-----+-----+
|  DIMMED  | -----+
| (15% BL) |
+-----+-----+
      | 5min keine Praesenz
      v
+-----+-----+
| LIGHT SLEEP | <-- Radar weckt per Interrupt
| (~0,8mA)    |      Aufwachzeit ~2ms
+-----+-----+

```

Light Sleep wurde Deep Sleep vorgezogen, da die schnellere Aufwachzeit (~ 2 ms vs. mehrere Sekunden) eine sofortige Reaktion auf Präsenzerkennung ermöglicht und Sensordaten im RAM erhalten bleiben.

3.6 Verwendete Libraries

Library	Version	Verwendung
LovyanGFX	1.1.16	Display-Treiber für ST7796S (SPI), Backlight-PWM
LVGL	9.2.2	UI-Framework, 5 Screen-Designs, Custom Fonts
Adafruit AHTX0	latest	AHT20 Temperatur/Feuchte-Sensor (I ² C)
Adafruit SGP40	latest	SGP40 VOC-Sensor mit VOC-Index-Algorithmus
MH-Z19	1.5.4	CO ₂ -Sensor Treiber
PMS Library	1.1.0	Feinstaub-Sensor Treiber (PM1.0/2.5/10)
ld2410	0.1.3	Radar-Sensor Treiber (Präsenz/Bewegung)
EspSoftwareSerial	8.2.0	Software-UART für MH-Z19C (GPIO4/5)

Änderungshistorie

Version	Datum	Autor	Änderung
1.0	Januar 2026	Team	Erstversion
2.0	Februar 2026	Team	LVGL 8.3→9.2.2. LovyanGFX 1.1.16 integriert. 5 Screens statt 2 UI-Modi. Deep Sleep durch Light Sleep ersetzt. Pin-Belegung aktualisiert. Modulare Code-Struktur dokumentiert. SoftwareSerial für MH-Z19C ergänzt.